MTP CMPSC 448, Identifying LLMs from Their Responses

Akseli Karsikas
October 2026

1. Introduction

Large language models are becoming really common, and different models can often give similar answers to the same questions. Though even when the information is similar, the way the models write their responses can be different. For this project, I wanted a neural network to learn these differences and identify which LLM generated a response.

I collected responses from three different LLMs: GPT-5.6 Luna, Gemini 3.5 Flash-Lite, and Qwen 3.8 27B. I then trained two different neural network models, a Convolutional Neural Network and a Long Short-Term Memory network to classify the responses.

Research questions:

- RQ1: Can a CNN or RNN identify which LLM generated a response?
- RQ2: Does using the input, output, or both affect classification accuracy?
- RQ3: Can a model trained on certain topics still identify LLMs when tested on completely different topics?
- RQ4: What differences in writing style and formatting help distinguish the LLMs?

My best CNN achieved 82.61% accuracy, while the LSTM achieved 65.22%. This showed that there were noticeable patterns in the responses that the neural networks could learn.

2. Dataset Collection

2.1 Collecting Responses

I created my own dataset by collecting responses from three different LLMs through their APIs.

LLM	Responses
GPT-5.6 Luna	150
Gemini 3.5 Flash-Lite	150
Qwen 3.8 27B	150
Total	450

I used Python scripts to send prompts to each model and save the responses into CSV files.

An important part of my dataset was that every LLM received the exact same 150 prompts, so that the models would be answering the same questions to help with making the comparison simpler. I combined everything into one dataset containing 450 responses.

2.2 Prompt Categories

I divided the prompts into five different categories.

Category	Prompts	Responses
Gymnastics	30	90
Random/general	30	90
Oulu	30	90
Finland	30	90
Academic	30	90
Total	150	450

I wanted the dataset to include different types of questions.

For example, gymnastics included questions about training and technique, while the Oulu and Finland categories focused more on geography and culture. The academic category included topics like mathematics and science.

I also included random questions to make the dataset more varied.

Each response was saved together with its prompt, prompt ID, category, and the name of the LLM that generated it.

After combining the files, I checked the dataset for missing values and duplicate responses. The final dataset had 450 valid responses, with 150 from each LLM.

2.3 Splitting the Dataset

I divided the dataset into training, validation, and testing sets.

Dataset	Prompts	Responses
Training	105	315
Validation	22	66
Testing	23	69

I used a random seed of 42 for reproducing purposes.

Instead of splitting individual responses randomly, I split the dataset using prompt IDs. This meant that all three responses to the same prompt stayed in the same dataset split, because otherwise the same question could appear in both training and testing, which could make the results misleading.

3. Models and Training

3.1 Preprocessing

Before training the models, I needed to convert the text responses into something the neural networks could process.

First, I converted the text to lowercase and separated it into tokens, including words, numbers, and punctuation.

I then created a vocabulary from the training data. Words appearing fewer than twice were excluded.

I also used two special tokens:

- `<PAD>` to make shorter sequences the same length.
- `<UNK>` for tokens that were not included in the vocabulary.

I limited each input to 300 tokens. Longer responses were cut off, and shorter ones were padded.

The output-only vocabulary contained 6,067 tokens, while the input-only vocabulary contained 488 tokens.

I made sure that the vocabulary was built using only training data so that information from the test set would not affect training.

3.2 CNN Model

The first model I implemented was a CNN.

My CNN used an embedding layer with 100 dimensions, followed by convolutional filters with kernel sizes of 3, 4, and 5.

I used 100 filters for each kernel size, followed by ReLU activation and max pooling.

After combining the outputs, I used dropout with a rate of 0.5 and a final linear layer to predict one of the three LLMs.

The main idea behind using a CNN was that it could recognize short patterns in the text.

For example, certain word combinations, punctuation patterns, or formatting styles might appear more often in responses from one LLM than another.

3.3 LSTM Model

The second model I implemented was an LSTM/RNN.

I used an embedding size of 100 and an LSTM hidden size of 128.

The model processed the text sequentially and used its final hidden state for classification. I also included dropout of 0.5 before the final classification layer.

I wanted to compare this model with the CNN because the LSTM processes information in order and can potentially learn patterns across longer sequences.

3.4 Training Settings

I trained both models using PyTorch.

Setting	Value
Optimizer	Adam
Learning rate	0.001
Loss function	Cross-entropy
Batch size	32
Training epochs	10
Random seed	42
Maximum sequence length	300

During training, I checked the validation accuracy after each epoch and saved the model with the best validation performance.

After training, I evaluated the saved models on the separate test dataset.

I used accuracy, precision, recall, F1-score, and confusion matrices to understand how well the models performed.

4. RQ1: Can Neural Networks Identify the LLM?

For my first experiment, I trained the CNN and LSTM using only the generated responses.

The goal was to see whether the models could predict which LLM wrote a response without seeing the original question.

4.1 Results

Model	Validation accuracy	Test accuracy	Macro F1
CNN	77.27%	82.61%	0.8214
LSTM	62.12%	65.22%	0.6498
Random guessing	33.33%	33.33%	—

Both models performed better than random guessing, but the CNN was clearly more accurate.

The CNN correctly classified 57 out of 69 test responses, while the LSTM correctly classified 45.

The difference between their accuracies was about 17.39 percentage points.

4.2 CNN Results

The CNN performed especially well when identifying GPT responses.

Here is the confusion matrix:

Actual / Predicted	GPT	Gemini	Qwen
GPT	23	0	0
Gemini	2	19	2
Qwen	1	7	15

The CNN correctly identified all 23 GPT responses.

However, it had more trouble distinguishing Gemini and Qwen. In particular, seven Qwen responses were incorrectly classified as Gemini.

This made me think that GPT might have more recognizable patterns in my dataset, while Gemini and Qwen may share some similar writing characteristics.

4.3 LSTM Results

The LSTM reached 65.22% test accuracy.

Its confusion matrix was:

Actual / Predicted	GPT	Gemini	Qwen
GPT	19	2	2
Gemini	1	13	9
Qwen	5	5	13

The LSTM made more mistakes, especially when distinguishing Gemini from Qwen.

I expected the LSTM to perform reasonably well because it processes the sequence of words, but the CNN ended up being much better.

One possible explanation is that the most useful differences between these LLMs were shorter patterns in their responses, which the CNN was better at recognizing.

However, this does not mean CNNs are always better than LSTMs. My dataset was relatively small, and the result also depends on how I implemented and trained the models.

5. RQ2: Input vs. Output vs. Both

For my second research question, I wanted to see how the information given to the classifier affected its accuracy.

I trained each model using three different input settings:

- Input only: Just the original prompt.
- Output only: Just the response generated by the LLM.
- Combined: The original prompt and generated response together.

5.1 Results

Model	Input only	Output only	Combined
CNN	33.33%	82.61%	81.16%
LSTM	33.33%	65.22%	53.62%

The results were pretty interesting.

Both models achieved exactly 33.33% accuracy when using only the input prompts.

This actually makes sense because I gave every LLM the same prompts. So, the input question alone does not contain information about which model generated the response.

The output-only setting performed the best for both architectures.

I originally thought combining the input and output might improve performance because it gives the classifier more context. However, that was not the case.

The CNN dropped slightly from 82.61% to 81.16%, while the LSTM dropped from 65.22% to 53.62%.

One possible reason is that I limited the sequences to 300 tokens. When the input and output were combined, part of the sequence was used by the prompt, leaving less room for the actual response.

The extra prompt information may also have made classification harder because it was not directly related to identifying the LLM.

Overall, this experiment showed that the generated response was much more useful than the input prompt for this task.

6. RQ3: Cross-Domain Generalization

For my third research question, I wanted to see whether the CNN was actually learning differences between the LLMs or just recognizing patterns related to specific topics.

To test this, I trained a new CNN using four of the five categories and then tested it on the category that was left out.

I repeated this experiment five times so that every category was used as the unseen test domain once.

6.1 Results

Unseen category	Accuracy
Gymnastics	80.00%
Random/general	75.56%
Oulu	86.67%
Finland	83.33%
Academic	63.33%
Average	77.78%

The average accuracy across the five experiments was 77.78%.

I thought this was one of the more interesting results because the CNN still performed well even when it had never seen responses about the test topic during training.

For example, when I trained the model without any Oulu-related responses, it still achieved 86.67% accuracy on that category.

The academic category was the hardest, with an accuracy of 63.33%.

One possible explanation is that academic questions encourage the LLMs to use more similar language and explanations, making them harder to distinguish. However, I would need more experiments to know whether that is actually the reason.

These results suggest that the CNN was learning some general characteristics of the different LLMs rather than only memorizing topic-specific words.

7. RQ4: Differences in LLM Writing Styles

For my fourth research question, I wanted to understand what made the responses from the three LLMs different.

I analyzed several characteristics, including response length, number of characters, lexical diversity, bullet points, bold text, and headings.

7.1 Response Length

LLM	Average words	Average characters	Lexical diversity
GPT	183.77	1,227.93	0.662
Gemini	529.08	3,376.78	0.520
Qwen	310.11	2,012.39	0.569

One of the biggest differences I noticed was response length.

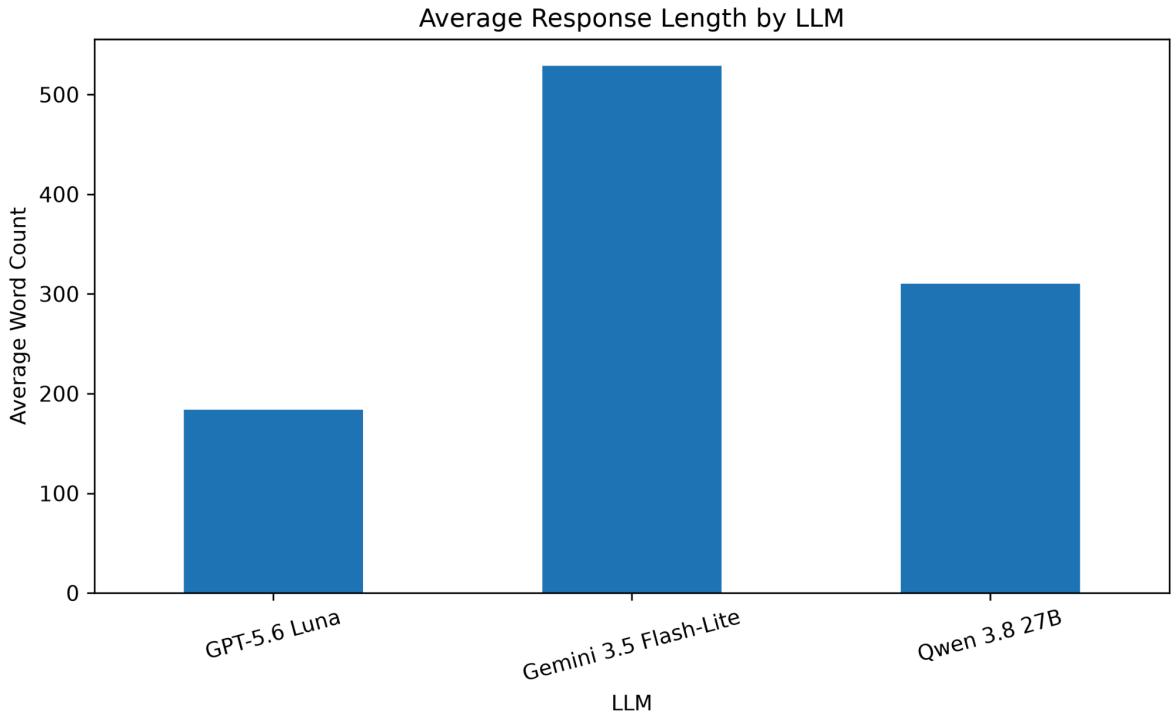
Gemini produced much longer responses on average, with about 529 words per response.

Qwen averaged around 310 words, while GPT averaged only about 184.

I also calculated lexical diversity by dividing the number of unique words by the total number of words.

GPT had the highest lexical diversity at 0.662, followed by Qwen and Gemini.

However, this measurement is affected by response length. Shorter responses naturally tend to have a higher proportion of unique words, so this does not necessarily mean GPT uses a more diverse vocabulary overall.



7.2 Formatting Differences

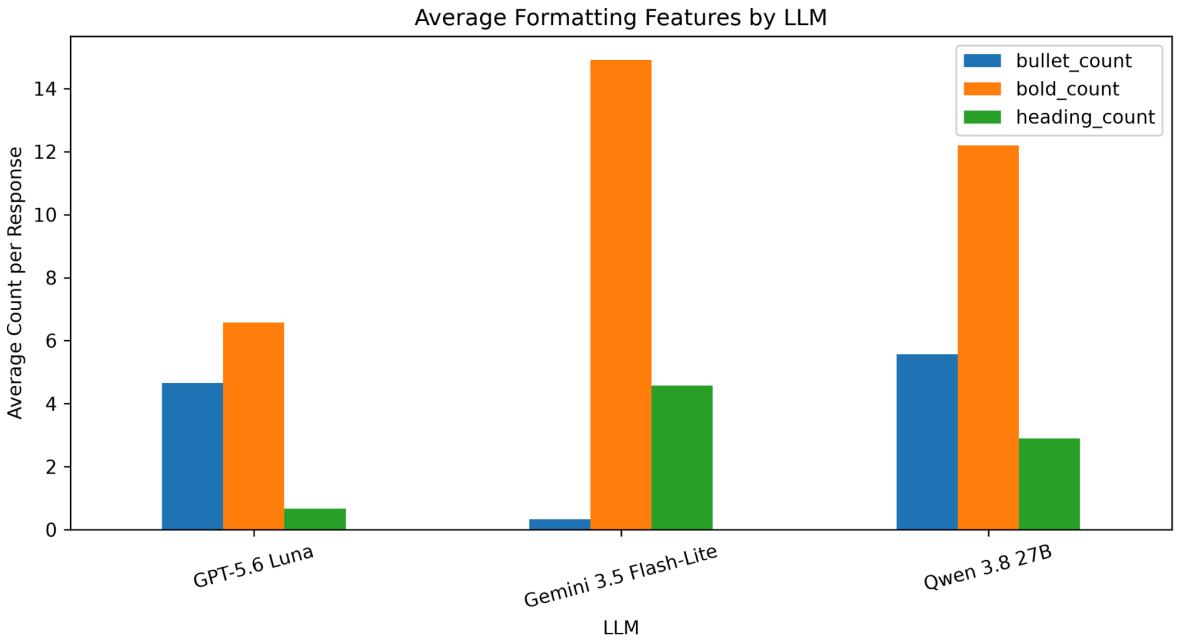
I also looked at how often each model used Markdown formatting.

LLM	Bullet markers	Bold spans	Headings
GPT	4.66	6.58	0.67
Gemini	0.33	14.91	4.58
Qwen	5.57	12.20	2.89

These numbers represent the average number of each formatting feature per response.

Gemini used significantly more headings and bold text than GPT, while Qwen used the most bullet markers.

GPT generally had fewer headings and shorter responses.



I think these formatting differences could help explain why the CNN performed so well.

Since CNNs can recognize local patterns in text it might give the classifier useful clues about which model generated the response.

However, I did not directly test whether removing these features would reduce accuracy, so I cannot say for sure how much the classifier relied on them.

7.3 What I Learned from the Analysis

Before doing this project, I expected the models to have somewhat different writing styles, but I did not expect the differences in response length and formatting to be this large.

The results helped me understand why a neural network might be able to identify an LLM even when the models are answering the same question.

At the same time, these are patterns from my particular dataset. Different prompts or API settings could produce different results.

8. Limitations

First, my dataset only contains 450 responses from three LLMs. Although this was enough to train and compare the classifiers, a larger dataset would make the results more reliable.

Second, I did not use completely identical generation settings across the three APIs. For example, I used a 500-token generation limit for GPT and Qwen, while Gemini did not have the same explicit limit in my collection script. This could have affected response length and made some models easier to identify.

Third, I limited the input sequences to 300 tokens. Since some responses were much longer than this, the classifiers did not always receive the full response.

This may also explain why combining the prompt and response did not improve accuracy.

Another limitation is how I handled padding in the LSTM. The model used the final hidden state after processing padded sequences, rather than explicitly stopping at the last real token. This could have affected its performance, especially for shorter responses.

Finally, my test set contained only 69 responses. This means that a relatively small number of incorrect predictions could noticeably change the final accuracy.

Because of these limitations, I would not assume that the same accuracy results would apply to every LLM or every type of prompt.

9. Project Experience and Lessons Learned

This project gave me a much better understanding of what goes into building a machine learning classifier from the beginning.

One of the first things I learned was how important the dataset is. Collecting responses from three different APIs took some work, and I had to make sure the responses were saved consistently so that I could combine them.

I also learned why the way data is split matters. Since every model answered the same prompts, I needed to split by prompt ID instead of randomly splitting individual responses.

Another thing I found interesting was that the CNN performed better than the LSTM. I initially thought that processing the text sequentially might be useful, but the CNN's ability to recognize shorter patterns seemed to work better for my dataset.

The input-versus-output experiment was also useful because it showed that giving a neural network more information does not automatically improve its performance.

I also learned that accuracy alone does not explain everything. Looking at confusion matrices helped me see which LLMs were being confused with each other, while analyzing response characteristics gave me possible explanations for those mistakes.

I used AI tools during the project to help me understand concepts, get ideas and debug code, and think through the results. I reviewed the implementation, ran the experiments, and checked the outputs myself. AI assistance was especially useful when troubleshooting parts of the data collection and model training process.

Overall, the project helped me connect concepts from class, such as CNNs, RNNs, embeddings, and classification, to an actual machine learning problem.

10. Conclusion

The goal of my project was to see whether neural networks could identify which LLM generated a response.

The results showed that this was possible with reasonable accuracy.

My CNN achieved 82.61% test accuracy, while the LSTM achieved 65.22%.

I also found that using only the generated response worked better than using the original prompt or combining the two.

The cross-domain experiments were particularly interesting because the CNN still achieved an average accuracy of 77.78% when tested on topics it had not seen during training.

Also, my analysis showed differences in response length, formatting, and lexical diversity across all of the AI's.

I learned that even when LLMs produce similar information, they can have recognizable patterns in how they write their responses. Neural networks can learn these patterns, although the results also depend on the dataset, model architecture, etc.

Appendix: Code and Reproducibility

GitHub Repository: https://github.com/AkseliKarsikas/CMPSC448-LLM-Identification

Programming language: Python

Main libraries: PyTorch, pandas, NumPy, scikit-learn, and Matplotlib

Dataset: 450 responses from three LLMs across five categories

Random seed: 42

Evaluation methods: Accuracy, precision, recall, F1-score, confusion matrices, cross-domain testing, and response-characteristic analysis

The GitHub repository contains my data collection scripts, preprocessing code, CNN and LSTM implementations, training and evaluation scripts, datasets, and experimental results.